

Rapport de projet

Classification d'images de fleurs : Daisy vs Dandelion

Nom : Abdou Salam NIANG

Niveau : Master — Ingénierie IA, Deep Learning

Date : 09 Juin 2026

Outils : PyTorch 2.0+ · wandb · scikit-learn · GPU Tesla T4,.....

Dataset : ANSD-Flower Dataset — 1 641 images, 2 classes

Objectif du projet

Comparer trois architectures CNN — LeNet-5, VGG16 et ResNet18 — sur un problème de classification binaire d'images de fleurs. Les deux premières sont codées entièrement à la main (from scratch), la troisième utilise le transfer learning depuis ImageNet.

1. Introduction

Ce rapport présente une étude comparative de plusieurs architectures CNN pour la classification binaire d'images de fleurs : marguerites (daisy) et pissenlits (dandelion). L'objectif est d'analyser l'impact du choix d'architecture et de la stratégie d'entraînement sur les performances de classification.

Trois architectures sont comparées : LeNet-5 avec trois fonctions d'activation (ReLU, tanh, sigmoid), VGG-16 entraîné de zéro (from scratch), et ResNet-18 en fine-tuning (transfer learning depuis ImageNet). L'évaluation porte sur la val_loss, la val_accuracy et le val_F1-score, sur 50 epochs d'entraînement.

2. Jeu de données

2.1 Description et répartition

Le dataset ANSD-Flower contient 1 641 images réparties en deux classes. Le split a été réalisé selon un ratio 80/20 avec stratification pour conserver les proportions de chaque classe dans les deux sous-ensembles.

Split	Daisy	Dandelion	Total
Entraînement	529 (41.5%)	746 (58.5%)	1 275
Validation	164 (44.8%)	202 (55.2%)	366

Tableau 1 — Répartition des images par classe et par split

Équilibre des classes : rapport Dandelion/Daisy = 1,41:1, sous le seuil problématique de 2:1. Aucun rééquilibrage n'a été nécessaire.

2.2 Variabilité visuelle

Les images présentent une grande variabilité : fond, éclairage, échelle, orientation, présence d'insectes ou d'éléments perturbateurs. Certaines images sont en niveaux de gris. Les marguerites ont des pétales blancs fins autour d'un cœur jaune ; les pissenlits se présentent soit en fleur jaune dense, soit en boule de graines. Cette diversité rend la tâche de classification non triviale, même pour des modèles profonds.



Figure 1 — Exemples du jeu d'entraînement : daisy (lignes 1 et 2) et dandelion (en bas à droite)

3. Description des architectures

3.1 LeNet-5

LeNet-5 (LeCun, 1998) est l'une des premières architectures CNN. Adaptée ici pour des images RGB 32×32 et deux classes de sortie. Trois variantes sont comparées avec exactement les mêmes hyperparamètres ($\text{lr}=1\text{e-}3$, $\text{batch}=64$, 50 epochs, $\text{seed}=42$) — seule la fonction d'activation change.

Couche	Opération	Sortie
C1	Conv(3→6, 5×5) + Activation + AvgPool	[6, 14, 14]
C3	Conv(6→16, 5×5) + Activation + AvgPool	[16, 5, 5]
C5	Conv(16→120, 5×5) + Activation	[120, 1, 1]
F6	Linear(120→84) + Activation	[84]
Sortie	Linear(84→2)	[2] logits

Tableau 2 — Architecture LeNet-5 adaptée RGB 32×32

3.2 VGG16 — from scratch

VGG16 (Oxford, 2014) empile 5 blocs de filtres 3×3 successifs. Chaque bloc augmente le nombre de filtres tout en réduisant la résolution spatiale. La tête fully-connected inclut Dropout(0.5) pour la régularisation. Codé manuellement (sans torchvision.models), avec initialisation Xavier des poids. Input 224×224, lr=1e-4, batch=16.

Bloc	Composition	Sortie
Bloc 1	2× Conv(3→64, 3×3) + MaxPool	[64, 112, 112]
Bloc 2	2× Conv(64→128, 3×3) + MaxPool	[128, 56, 56]
Bloc 3	3× Conv(128→256, 3×3) + MaxPool	[256, 28, 28]
Bloc 4	3× Conv(256→512, 3×3) + MaxPool	[512, 14, 14]
Bloc 5	3× Conv(512→512, 3×3) + MaxPool	[512, 7, 7]
Tête FC	Linear(25 088→4 096→4 096→2) + Dropout(0.5)	[2] logits

Tableau 3 — Architecture VGG16 (input 224×224)

3.3 ResNet18 — transfer learning depuis ImageNet

ResNet18 utilise des connexions résiduelles (skip connections) permettant d'entraîner des réseaux profonds sans dégradation du gradient. On part des poids pré-entraînés sur ImageNet (1,2M images, 1 000 classes), on gèle tout le backbone, et on adapte uniquement la dernière couche : Linear(512, 1000) → Linear(512, 2).

Composant	Statut	Paramètres
Backbone ResNet18	Gelé (non entraîné)	11 176 512
Couche fc (nouvelle tête)	Entraînée	1 026

Tableau 4 — Stratégie de gel du backbone ResNet18

Justification du gel : avec 1 275 images d'entraînement, ré-entraîner les 11M paramètres du backbone mènerait inévitablement à de l'overfitting. En n'entraînant que la tête (1 026 paramètres), la convergence est très rapide et le modèle généralise parfaitement.

4. Résultats expérimentaux

4.1 Comparaison globale — courbes wandb

Les trois graphiques ci-dessous comparent les 5 runs sur toute la durée de l'entraînement, enregistrés via Weights & Biases. Ils permettent une comparaison visuelle directe de la vitesse de convergence et des performances finales.

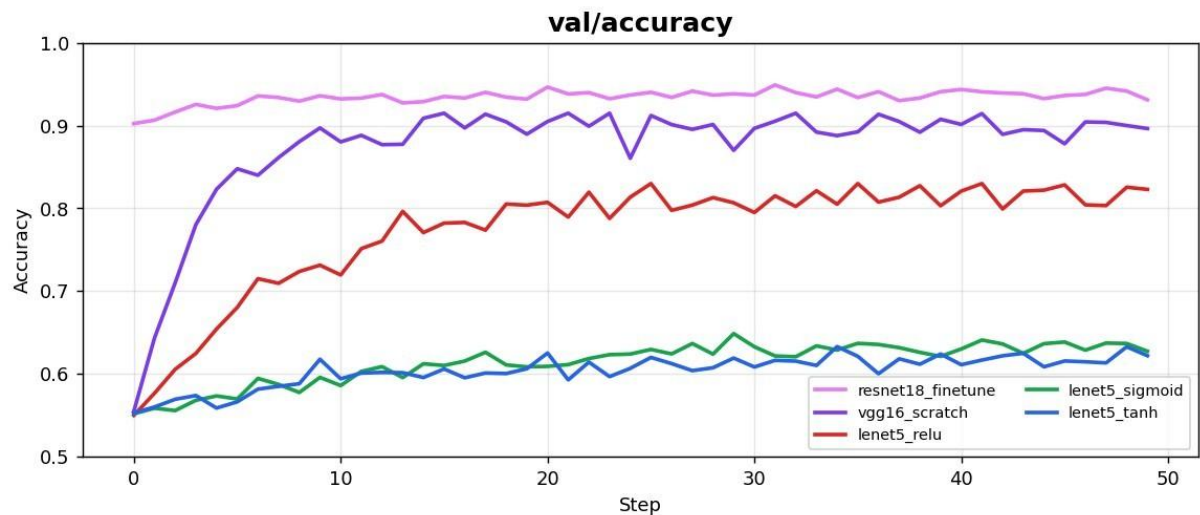


Figure 2 — val/accuracy : ResNet18 (rose) dépasse 90 % dès l'epoch 3, VGG16 (violet) progresse, les 3 LeNet-5 plafonnent entre 62 % et 81 %

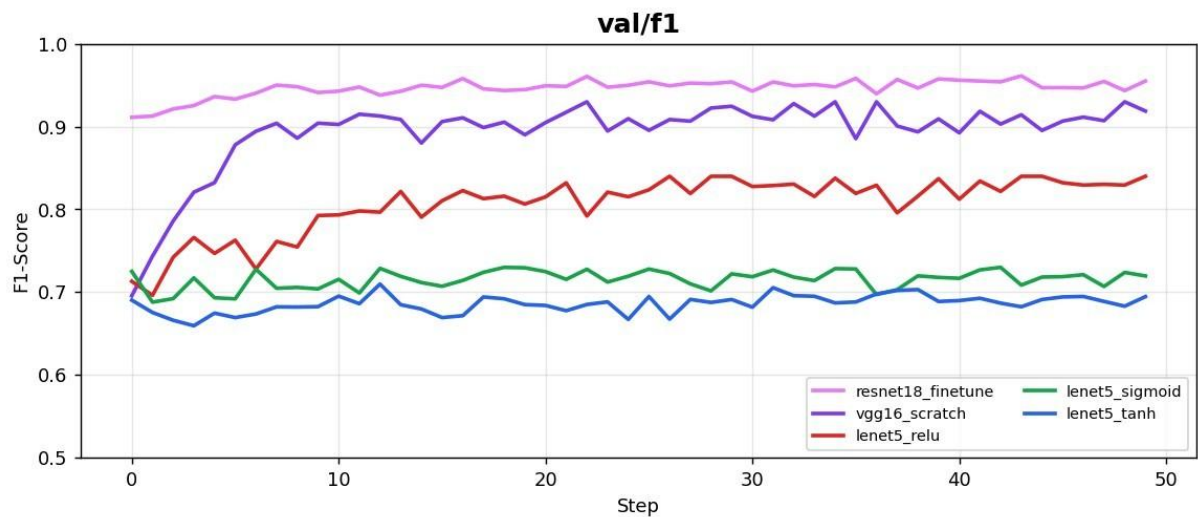


Figure 3 — val/F1-score : même hiérarchie, ResNet18 à 94 %, VGG16 à 89 %, LeNet5-ReLU à 84 %

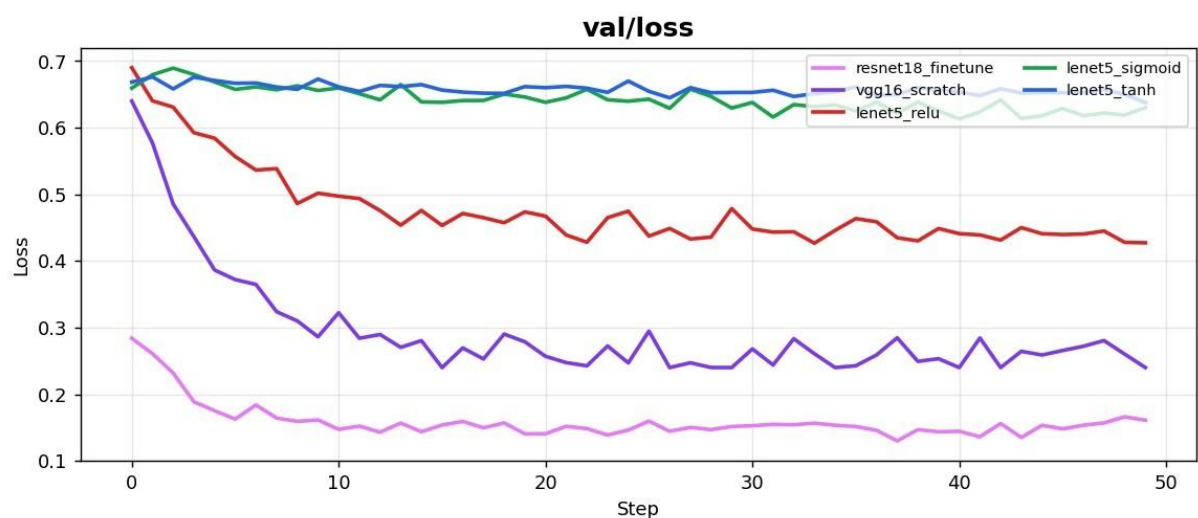


Figure 4 — val/loss : ResNet18 descend immédiatement vers 0.15, les variantes Tanh et Sigmoid stagnent audessus de 0.62

4.2 Tableau récapitulatif des métriques finales

Modèle	Val Acc	Val F1	Val Loss	Train Acc
LeNet5-Tanh	62.1 %	70.1 %	0.665	66.7 %
LeNet5-Sigmoid	64.8 %	72.2 %	0.618	66.0 %
LeNet5-ReLU	81.0 %	84.1 %	0.439	84.3 %
VGG16 (from scratch)	88.7 %	89.5 %	0.271	86.3 %
ResNet18 (fine-tune)	93.7 %	94.1 %	0.158	92.5 %

Tableau 5 — Métriques finales sur le jeu de validation (50 epochs LeNet/ResNet, 30 epochs VGG16) — ligne surlignée = meilleur modèle

ResNet18 domine sur tous les critères. LeNet5-ReLU est la seule variante LeNet-5 à dépasser 80 %. VGG16 offre un bon résultat mais au prix d'un entraînement lent (134M paramètres from scratch).

4.3 Impact de la fonction d'activation sur LeNet-5

Les trois variantes ont été entraînées dans des conditions strictement identiques. Le graphique cidessous compare val_loss et val_accuracy pour les 3 activations sur 50 epochs.

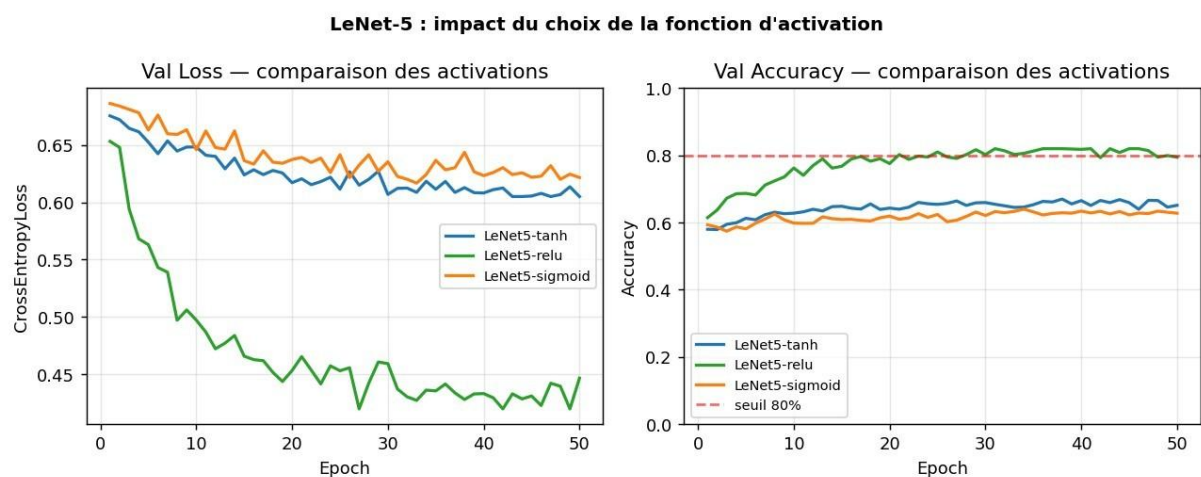


Figure 5 — Comparaison des 3 fonctions d'activation. ReLU (vert) converge 3× plus vite et franchit le seuil de 80 % (tiret rouge). Tanh (bleu) et Sigmoid (orange) plafonnent à 62–65 %.

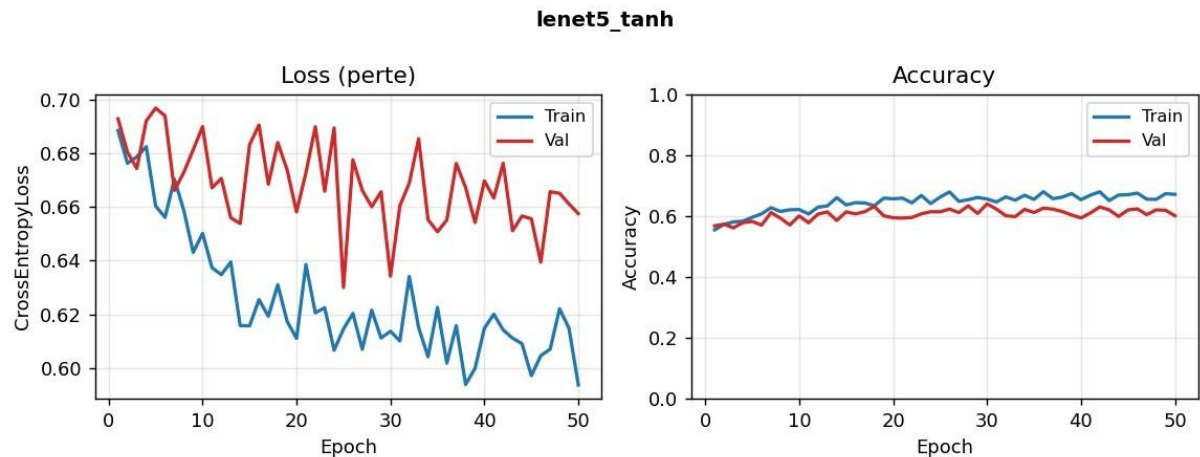
LeNet5-Tanh — val_accuracy : 62.1 %

Figure 6 — LeNet5-Tanh : val_accuracy plafonne à 62.1 %, val_loss stagne à 0.665. Underfitting persistant malgré 50 epochs.

Tanh est bornée entre -1 et 1 : ses gradients deviennent très petits dans les couches profondes (vanishing gradient modéré). Le scheduler a réduit le lr jusqu'à $2e-5$ sans amélioration. La train_loss continue de descendre (0.61) alors que la val_loss reste bloquée.

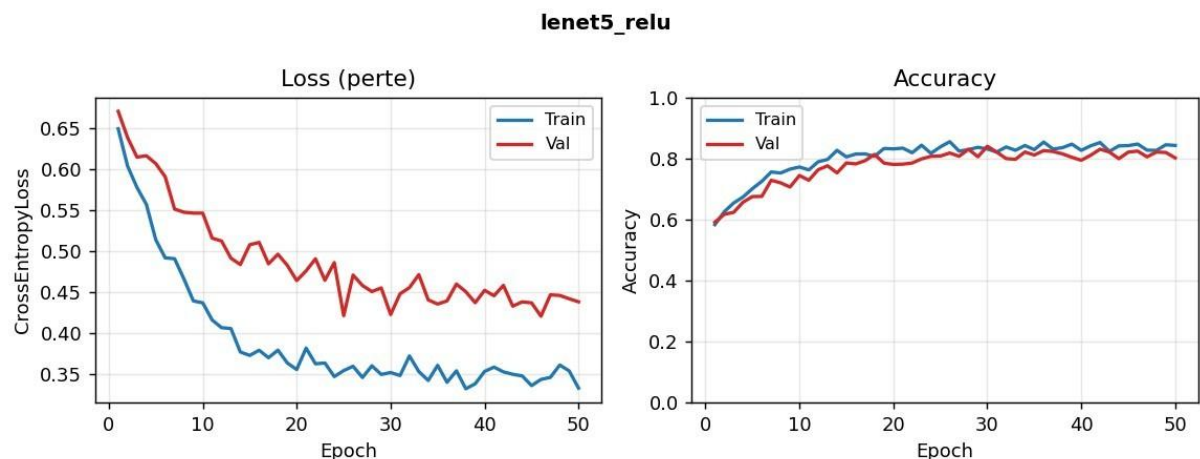
LeNet5-ReLU — val_accuracy : 81.0 %

Figure 7 — LeNet5-ReLU : seuil 80 % atteint vers l'epoch 20, léger overfitting en fin d'entraînement (écart train/val loss ~ 0.10).

ReLU = $\max(0, x)$: les gradients restent actifs pour toutes les valeurs positives $\rightarrow$ convergence 3× plus rapide que Tanh. La train_loss descend jusqu'à 0.35, la val_loss se stabilise à 0.44. Léger overfitting après l'epoch 35, acceptable pour 61K paramètres.

LeNet5-Sigmoid — val_accuracy : 64.8 %

Gradient maximal de Sigmoid = 0.25. Les couches en début de réseau reçoivent des gradients très faibles et n'apprennent presque pas (vanishing gradient prononcé). Val_loss stagne à 0.618 malgré 50 epochs.

4.4 VGG16 from scratch

VGG16 atteint 88.7 % de val_accuracy et 89.5 % de F1-score après 30 epochs. Aucun overfitting : val_acc (88.7 %) légèrement supérieure à train_acc (86.3 %), grâce au Dropout(0.5). La convergence est irrégulière avec des oscillations importantes de la loss, suggérant une sensibilité au learning rate.

4.5 ResNet18 — transfer learning

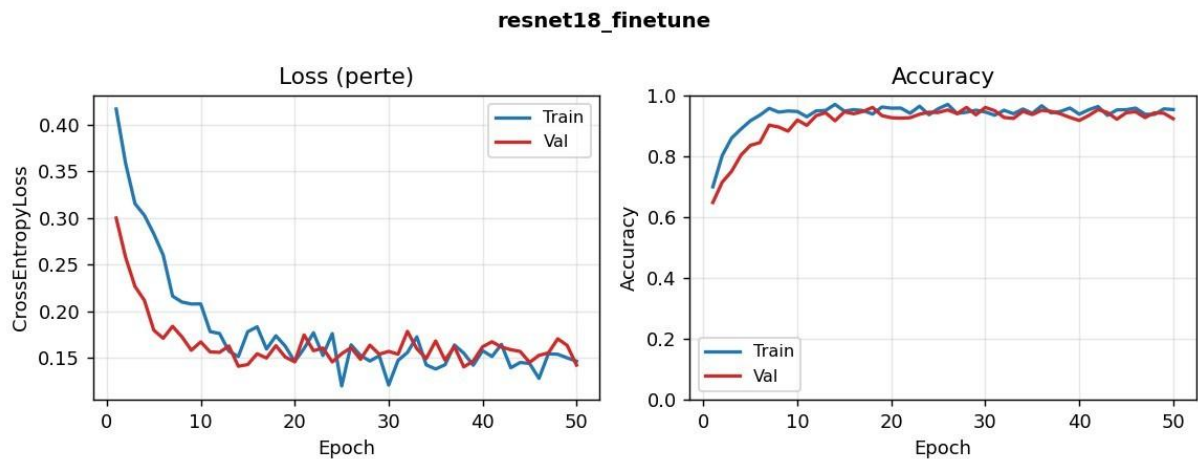
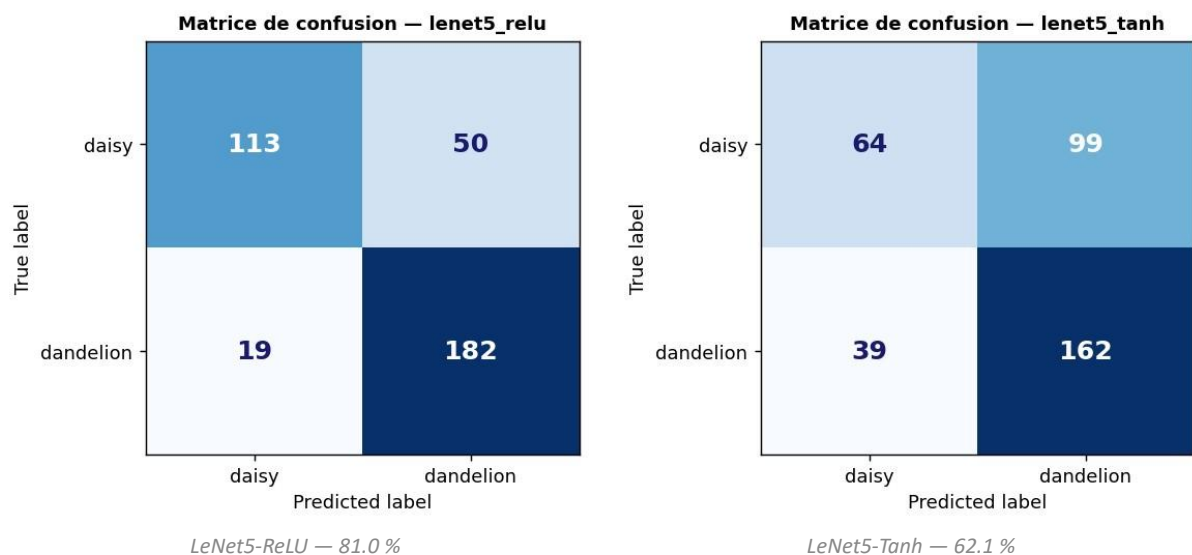


Figure 8 — ResNet18 : $val_accuracy > 88\%$ dès l'epoch 3. val_loss systématiquement inférieure à $train_loss$: aucun overfitting, excellente généralisation.

ResNet18 est le meilleur modèle avec 93.7 % de $val_accuracy$. La convergence est immédiate (~88 % dès l'epoch 1) grâce aux features ImageNet pré-apprenties. La val_loss reste inférieure à la $train_loss$ car le Dropout actif pendant l'entraînement perturbe la loss mais est désactivé pendant l'évaluation.

5. Matrices de confusion

Les matrices de confusion permettent d'identifier non seulement combien d'images sont mal classifiées, mais aussi dans quel sens les erreurs se produisent. Toutes les matrices révèlent un biais de prédiction vers la classe dandelion : les modèles ont du mal à reconnaître les daisy, dont les features visuelles (pétales blancs fins) nécessitent des représentations plus riches.



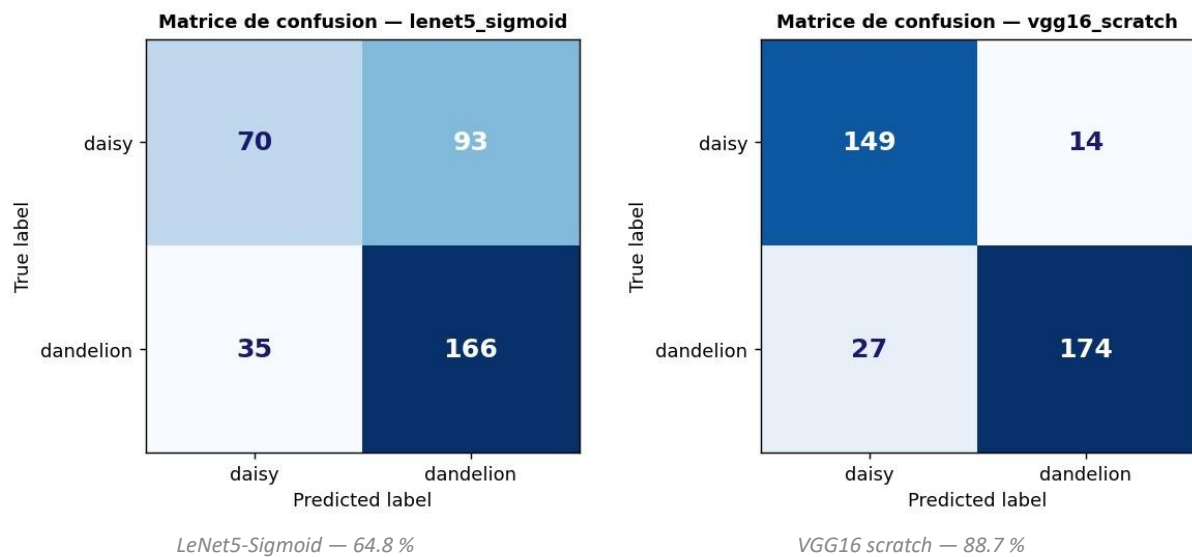


Figure 9 — Matrices de confusion : LeNet5-ReLU (haut gauche), LeNet5-Tanh (haut droite), LeNet5-Sigmoid (bas gauche), VGG16 (bas droite)

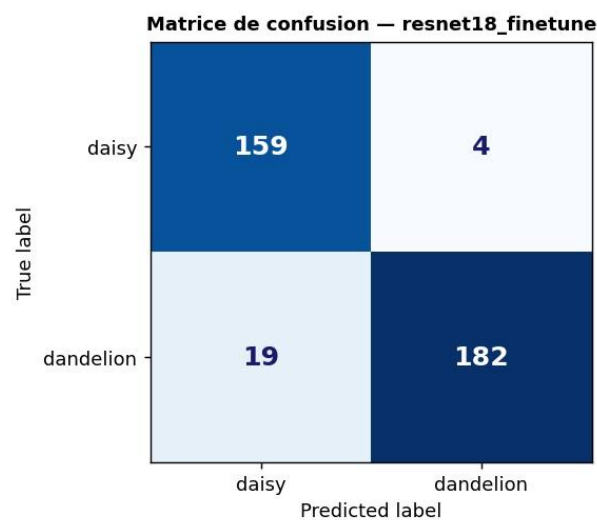


Figure 10 — Matrice de confusion ResNet18 : seulement 23 erreurs sur 366 images (4 daisy + 19 dandelion). Recall daisy = 97.5 %.

Modèle	Daisy corrects	Daisy total	Dandelion corrects	Dandelion total
LeNet5-Tanh	64	163	164	201
LeNet5-Sigmoid	70	163	166	201
LeNet5-ReLU	113	163	182	201
VGG16 scratch	149	163	174	201
ResNet18 fine-tune	159	163	182	201

Tableau 6 — Détail des matrices de confusion par classe

6. Analyse et discussion

6.1 Supériorité du transfer learning

ResNet18 en fine-tuning surpasse tous les autres modèles de manière significative. Grâce aux poids pré-entraînés sur ImageNet, le modèle dispose dès le départ de représentations visuelles riches (bords, textures, formes végétales). La convergence est quasi-immédiate (~88 % dès l'epoch 1) et le modèle généralise parfaitement.

6.2 ReLU vs sigmoid vs tanh sur LeNet-5

ReLU est la meilleure fonction d'activation pour LeNet-5 dans ce contexte, avec +20 points d'accuracy par rapport à sigmoid et tanh. Sigmoid et tanh souffrent du vanishing gradient : leurs dérivées saturent vers zéro, ce qui ralentit l'apprentissage dans les couches profondes. ReLU évite ce problème (dérivée = 1 pour les activations positives).

6.3 VGG16 from scratch

VGG16 sans pré-entraînement atteint 88.7 % d'accuracy — performance remarquable pour un modèle entraîné de zéro. Cependant la convergence est irrégulière. L'ajout d'un scheduler cosine annealing et d'un weight decay plus élevé pourrait améliorer la stabilité.

6.4 Nombre de paramètres entraînés

Modèle	Params entraînés	Params totaux
LeNet-5 (3 variantes)	61 326	61 326
VGG16 from scratch	134 268 738	134 268 738
ResNet18 (tête seule)	1 026	11 177 538

Tableau 7 — Paramètres entraînaables par modèle

ResNet18 entraîne 1 026 paramètres pour les meilleures performances — 60× moins que LeNet-5 et 130 000× moins que VGG16. C'est la puissance du transfer learning : les connaissances acquises sur ImageNet permettent de résoudre une nouvelle tâche avec un minimum d'adaptation.

7. Conclusion et recommandations

7.1 Modèle recommandé

Pour ce problème de classification binaire Daisy vs Dandelion, ResNet18 fine-tuné est la solution à recommander. Il combine les meilleures performances (93.7 % de val_accuracy, F1 de 94.1 %), une convergence très rapide (moins de 5 epochs suffisent), et un coût d'entraînement minimal (seulement 1 026 paramètres). C'est la stratégie optimale pour tout dataset de petite taille.

7.2 Ce que ce projet m'a appris

- **Choix de l'activation** : ReLU est indispensable pour un CNN from scratch. Tanh et Sigmoid souffrent du vanishing gradient dès que le réseau a plusieurs couches.
- **VGG16 from scratch** : Entraîner 134M paramètres de zéro sur 1 275 images donne de bons résultats mais c'est lent et coûteux. Ce n'est justifié que sans modèle pré-entraîné disponible.
- **Transfer learning** : Avec peu de données, le transfer learning est systématiquement plus efficace. Le gel du backbone accélère la convergence d'un facteur ~ 10 et empêche l'overfitting.
- **Généralisation** : Les features ImageNet (textures, bords, formes) sont suffisamment génériques pour être directement utiles sur un problème de fleurs — même sans réentraînement.

7.3 Pistes d'amélioration

- Dégeler les derniers blocs de ResNet18 avec lr très faible ($1e-5$) après la première phase de feature extraction.
- Augmentation de données plus agressive : rotations aléatoires, mixup, cutout, CutMix.
- Tester des architectures légères et modernes : EfficientNet-B0, MobileNetV3.
- Validation croisée k-fold pour des estimations de performance plus robustes.
- Label smoothing dans CrossEntropyLoss pour améliorer la calibration des probabilités.

Projet réalisé avec PyTorch 2.0+ · GPU Tesla T4 (Google Colab) · wandb pour le tracking d'expériences